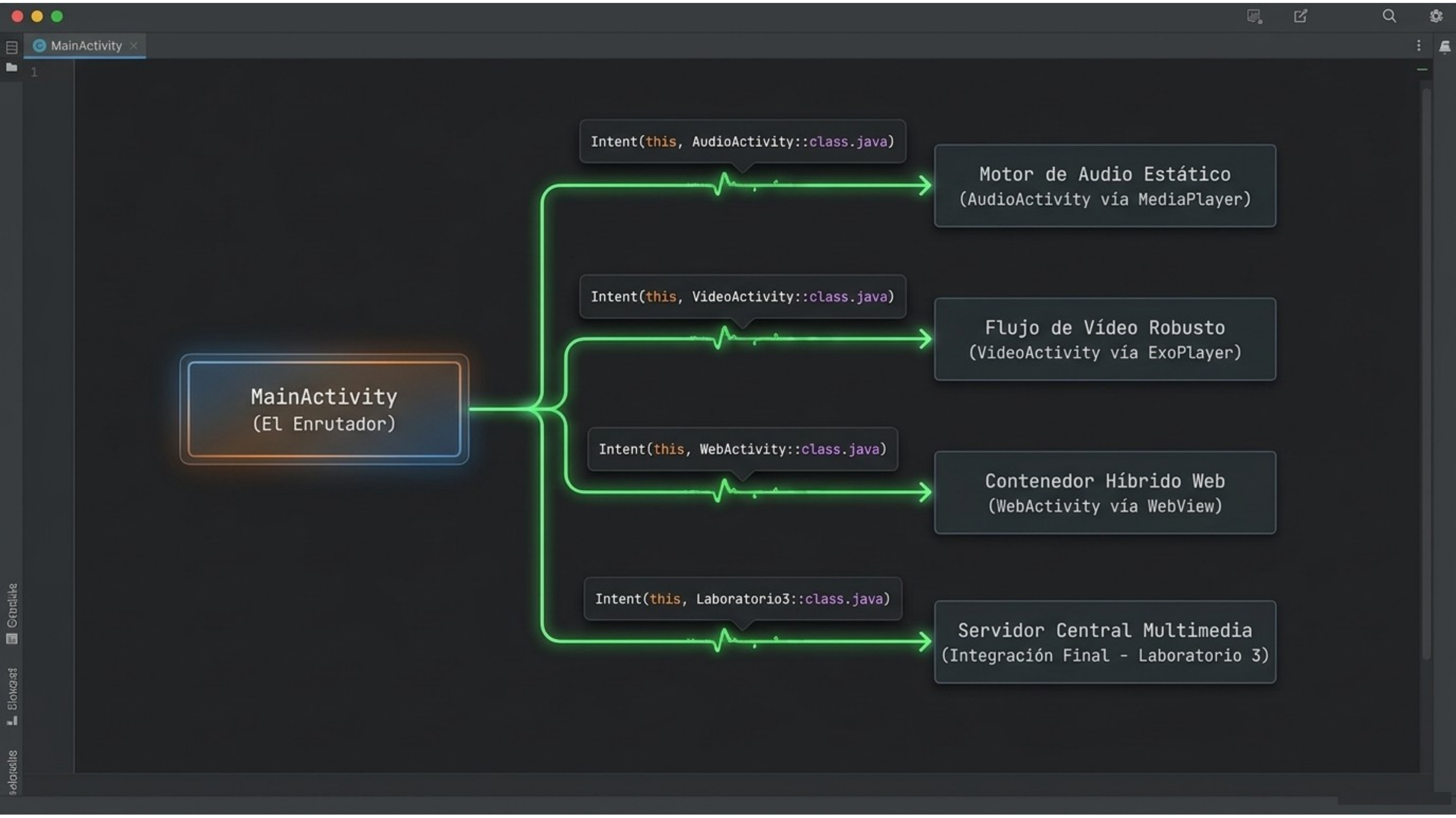
```kotlin
fun main() {
    buildMultimediaSystem()
}
```

```kotlin
// Programación Multimedia Avanzada en Kotlin
// Arquitectura de ExoPlayer, Concurrencia de Estado y Vinculación de Interfaz.
```


MainActivity
1
MainActivity
(El Enrutador)
Intent(this, AudioActivity::class.java)
Motor de Audio Estático
(AudioActivity vía MediaPlayer)
Intent(this, VideoActivity::class.java)
Flujo de Vídeo Robusto
(VideoActivity vía ExoPlayer)
Intent(this, WebActivity::class.java)
Contenedor Híbrido Web
(WebActivity vía WebView)
Intent(this, Laboratorio3::class.java)
Servidor Central Multimedia
(Integración Final - Laboratorio 3)

activity_main.xml



Enrutamiento Explícito Basado en Eventos

1 Instanciación

Separación: XML para estructura, Kotlin para lógica.

```
val btnIrVideo = findViewById<Button>(R.id.btnIrVideo)
```

2 Oyente de Eventos

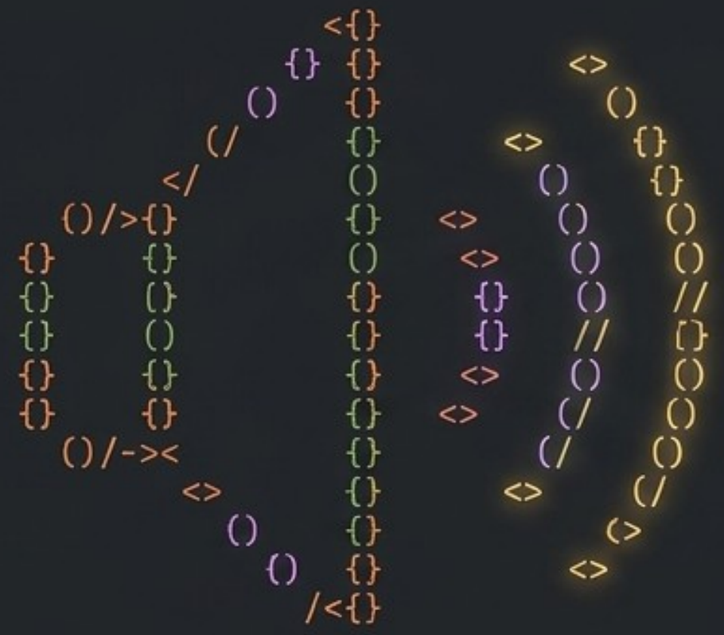
El disparador asíncrono.

```
.setOnClickListener { ... }
```

3 Transición de Contexto

Ejecución del Intent explícito.

```
startActivity(intent)
```



Recurso Integrado

Carga directamente desde la carpeta inmutable `res/raw`.

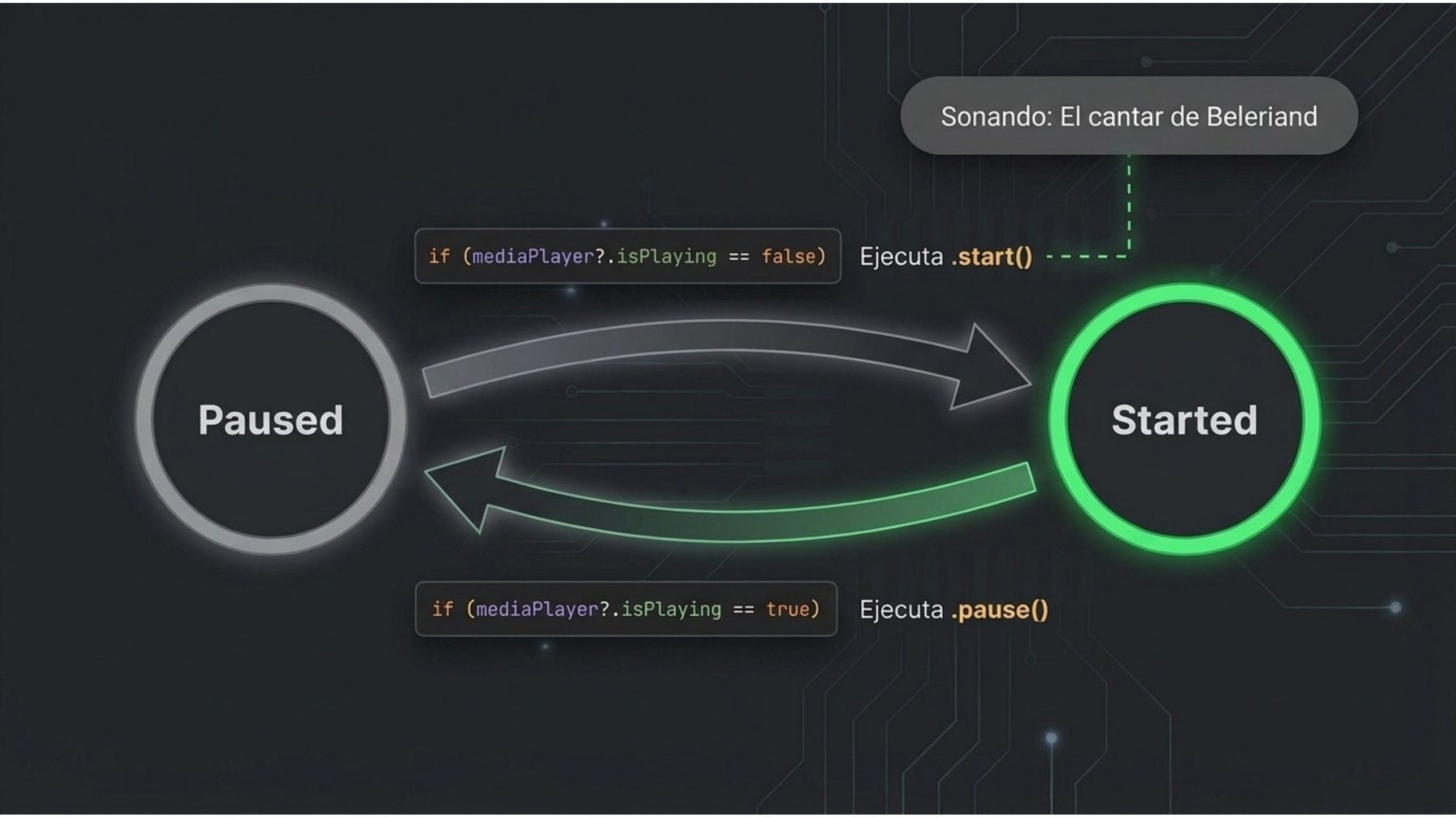
No requiere permisos de red.

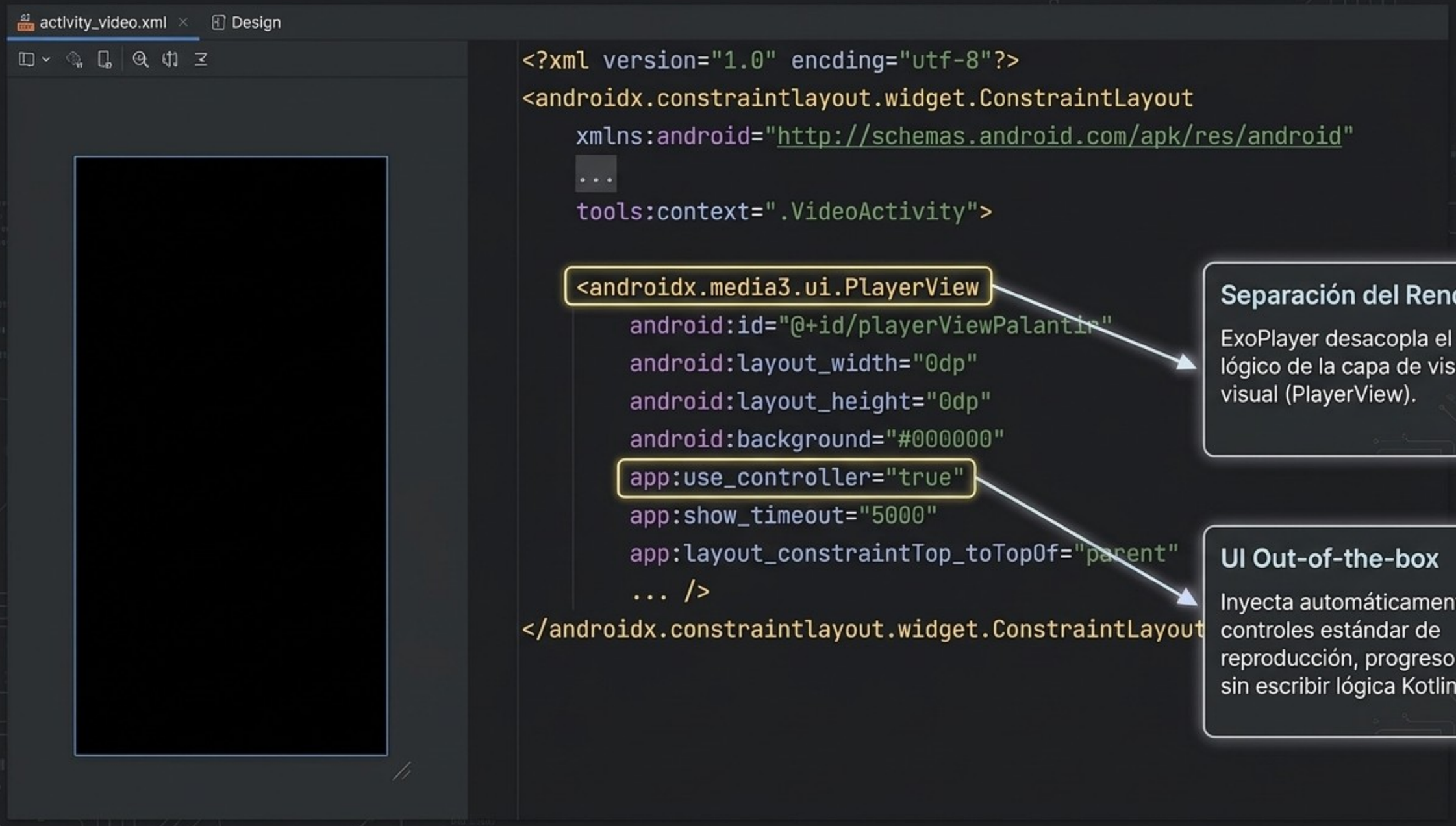
```
mediaPlayer = MediaPlayer.create(context = this, resid = R.raw.laud_finrod)
```

Operación Síncrona

Ideal para archivos locales ligeros o sonidos de interfaz de respuesta ultrarrápida.

Ineficiente para archivos masivos.


Sonando: El cantar de Beleriand
if (mediaPlayer?.isPlaying == false)
Ejecuta .start()
Paused
Started
if (mediaPlayer?.isPlaying == true)
Ejecuta .pause()


actlvity_video.xml
Design
<?xml version="1.0" encding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
...
tools:context=".VideoActivity">
<androidx.media3.ui.PlayerView
android:id="@+id/playerViewPalantir"
android:layout_width="0dp"
android:layout_height="0dp"
android:background="#000000"
app:use_controller="true"
app:show_timeout="5000"
app:layout_constraintTop_toTopOf="parent"
... />
</androidx.constraintlayout.widget.ConstraintLayout
Separación del Renderizado
ExoPlayer desacopla el motor lógico de la capa de visualización visual (PlayerView).
UI Out-of-the-box
Inyecta automáticamente los controles estándar de reproducción, progreso y pausa sin escribir lógica Kotlin adicional.

```
ExoPlayer.Builder(this).build()
```

1

Patrón Builder:

Construcción del motor lógico, optimizado para memoria y manejo de búfer dinámico.

```
playerView.player = exoPlayer
```

2

Vinculación Bidireccional:

Se ancla el motor (cerebro) al contenedor UI (músculo).

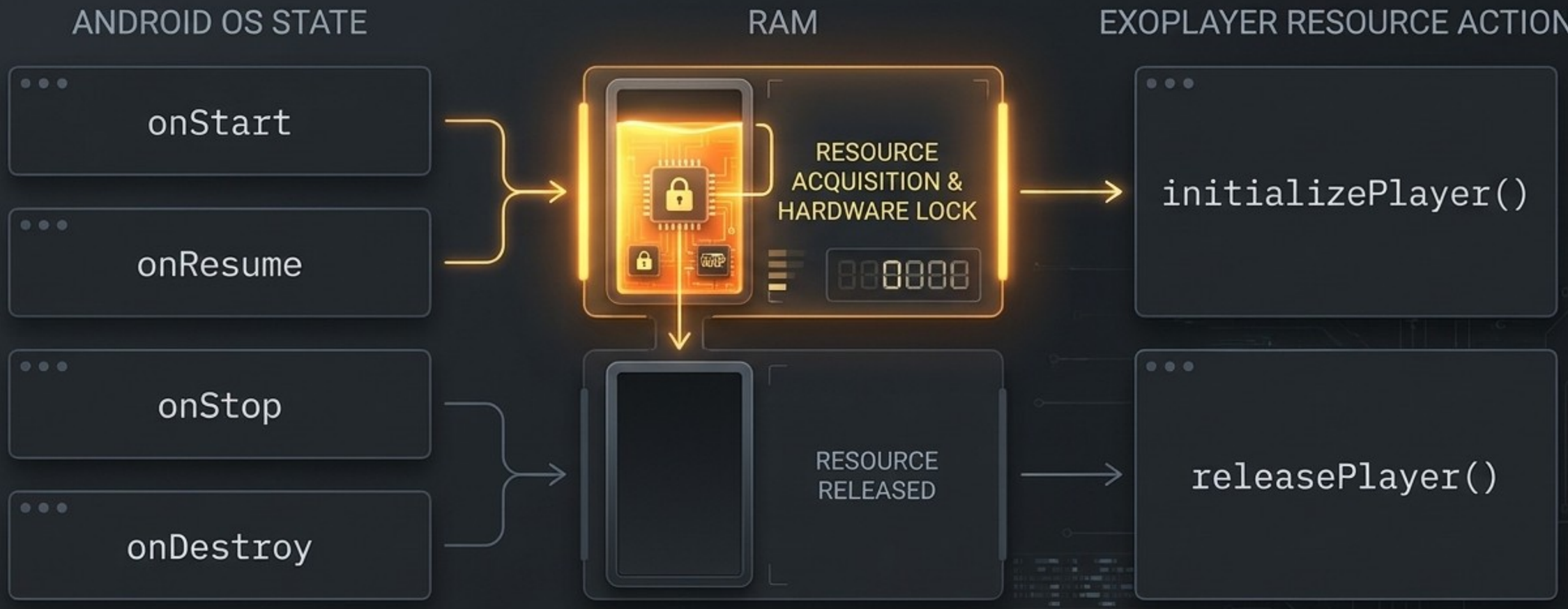
```
MediaItem.fromUri(...)
    .prepare()
    playWhenReady = true
```

3

Inyección y Disparo:

Se parsea el recurso, se precargan fragmentos en búfer y se ordena el inicio automático en cuanto el hardware esté listo.

ACTIVATION TIMELINE



OMITIR RELEASE() Y PLAYER = NULL CAUSARÁ FUGAS DE MEMORIA SEVERAS E IMPEDIRÁ QUE OTRAS APLICACIONES USEN EL DECODIFICADOR DE VÍDEO DEL TELÉFONO.




```kotlin
14    @SuppressLint("SetJavaScriptEnabled")
15    override fun onCreate(savedInstanceState: Bundle?) {
16        super.onCreate(savedInstanceState)
17        setContentView(R.layout.activity_web)
18
18        webView = findViewById(R.id.webViewPalantir)
19
20
21        webView.webViewClient = WebViewClient()
22        webView.webChromeClient = WebChromeClient()
23
24
25        webView.settings.javaScriptEnabled = true
26        webView.settings.domStorageEnabled = true
27        webView.settings.loadWithOverviewMode = true
28        webView.settings.useWideViewPort = true
```

Concesión de Riesgos Calculados: Bypass de la advertencia del IDE. Requerido obligatoriamente para ejecutar el reproductor embebido de YouTube, asumiendo la responsabilidad de seguridad XSS.

Permite al reproductor web almacenar cachés locales (resolución de video, volumen).

Ajusta la ventana gráfica del DOM al ancho físico del dispositivo móvil.

INJECTOR BOX

```
val iframeHtml = """
    <html>
    <head>
        <style>
            body { margin: 0; padding: 0; background-color: #000000; }
            .video-container { position: relative; width: 100%; height: 100%; }
            iframe { position: absolute; top: 0; left: 0; width: 100%; height: 100%; }
        </style>
    </head>
    <body>
        <div class="video-container">
            <iframe src="https://youtube.com" frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
        </div>
    </body>
    </html>
""".trimIndent()
```

El Contenedor (CSS): Garantiza un diseño responsivo móvil (Mobile First dentro del WebView).

```
webView.loadDataWithBaseURL("https://www.youtube.com", data = iframeHtml, mimeType = "text/html", encoding = "utf-8", historyUrl = null)
```

La Inyección de Código: Carga la estructura cruda directamente en memoria.

El Tipo MIME: El puente semántico que le dice al OS Android cómo interpretar la cadena de texto crudo.

MATRIZ COMPARATIVA: DIAGNÓSTICO DEL SISTEMA

MÉTRICA TÉCNICA	MediaPlayer	ExoPlayer	WebView
Origen de Datos	Local/Empaquetado	Red/Streaming Directo	Embebido de Terceros
Uso de Recursos (RAM)	🛡️ Muy Bajo	Moderado-Alto	⚠️ Muy Alto (Carga motor Chromium)
Soporte HLS / DASH (Streaming Adaptativo)	❌ No	✅ Sí, nativo	Depende del Player Web
Complejidad de Implementación	Línea Única	Patrón Builder/Ciclo de Vida	Inyección HTML/JS